



Sistemi Operativi

Mattia Cosimi

October 2024

1 Introduzione ai sistemi operativi

Un programma in esecuzione esegue istruzioni.
Il processore recupera un'istruzione dalla memoria,
La decodifica,
La esegue,
Successivamente si sposta sulla prossima istruzione (Modello di Von Neumann).

1.1 Di cosa si occupa il sistema operativo?

Il sistema operativo è responsabile di rendere semplice l'esecuzione di programmi, permettere ai programmi di condividere la memoria, permettere ai programmi di interagire con gli altri dispositivi. Il sistema operativo ha il compito di controllare che il sistema lavori in modo efficiente e corretto. Il sistema operativo prende una risorsa fisica e la trasforma in una virtuale. Sostanzialmente grazie alle chiamate di sistema permettiamo all'utente di dire cosa fare al sistema operativo. Quindi un sistema operativo gestisce le risorse, condivisione della cpu e condivisione delle memorie. Il sistema operativo virtualizzando la cpu ci dà l'impressione che questa utilizzi un numero infinito di cpu (Guarda cpu.c). Per quanto riguarda la virtualizzazione della memoria la memoria viene letta: specificando un indirizzo dove accedere ai dati e viene scritta: specificando i dati da scrivere all'indirizzo evocato (Guarda mem.c). Nella virtualizzazione della memoria ogni processo accede al suo spazio di indirizzi virtuali privati. Il sistema operativo mappa lo spazio degli indirizzi sulla memoria fisica. La memoria fisica è una risorsa condivisa gestita dal sistema operativo. Un altro problema è quello della concorrenza dato dai programmi multi thread. Si può immaginare che il nostro programma segua un filo principale e che durante l'esecuzione di possano creare dei nuovi fili secondari (Guarda threads.c). Quando in un programma abbiamo il semplice incremento di una variabile, in realtà stiamo eseguendo tre istruzioni: caricare il valore della variabile dalla memoria al registro, incrementarla e rimetterla nella memoria; Queste tre istruzioni non vengono eseguite atomicamente, vengono infatti considerate non un'unica istruzione ma tre diverse. Persistenza, hardware e software devono essere in grado di contenere dati persistentemente (file, dati...), Il sistema operativo si occupa di capire dove andranno messi i nuovi dati, infatti emette delle richieste al dispositivo di archiviazione sottostante. I goal del sistema operativo sono quindi: costruire un'astrazione e quindi rendere il sistema semplice da utilizzare, fornire delle buone performance, protezione tra le applicazioni e essere affidabile.

1.2 Il processo di astrazione

Processo: applicazione in esecuzione (Un'applicazione è un insieme di istruzioni su un qualche dispositivo di archiviazione). Il meccanismo che ci permette di eseguire più programmi con poche cpu è il time sharing, consiste nell'assegnare un'unità di tempo (quanto) a un processo, quando questa porzione di tempo finisce ho due opzioni 1. Finisco il processo che stavo eseguendo e passo a un altro 2. Non sono riuscito a finire il processo lo blocco momentaneamente per passare a un altro. Il meccanismo di stoppare un processo e passare a un altro viene detto context switching. Ovviamente il context switching è basato su alcune policy che prendono la decisione su quando stoppare un processo per quanto ecc... Tornando ai processi un processo è composto da la parte della memoria come istruzioni e dati e la parte dei registri come program counter e stack pointer. Il sistema operativo si occupa di controllare il ciclo di vita dei processi, lo fa con le seguenti funzioni: lo crea, lo distrugge, può metterlo in

stato di wait, gli fa i vari controlli e ottiene informazioni sullo stato del processo. Riguardo invece alla creazione di un processo: viene caricato il codice di un programma nella memoria, nello spazio degli indirizzi di un processo, si dice che il sistema operativo carica i processi in maniera “pigra” (Per ottimizzare l'utilizzo della memoria si carica il minimo indispensabile nella memoria e poi quando le funzioni vengono richieste viene allocata ulteriore memoria). Viene allocato lo stack a tempo di esecuzione, solitamente lo stack è utilizzato per variabili locali, parametri di funzioni e return address. Viene creato l'heap del programma, utilizzato per i dati allocati dinamicamente (malloc()), vengono inizializzate le altre cose e successivamente viene fatto eseguire il programma. Un processo può avere tre stati: in esecuzione (running), pronto (ready): il processo è pronto per andare in esecuzione ma per qualche motivo il sistema operativo ha deciso di non eseguirlo in questo momento, bloccato (blocked). Il sistema operativo ha delle strutture dati per organizzare le informazioni es. xv6 kernel proc structure. Ritornando al time sharing l'obiettivo del sistema operativo è quello di realizzare il context switching con meno overhead possibile, ovvero tutto quell'insieme di istruzioni che posso evitare di eseguire. Un altro obiettivo è quello di avere, da parte del SO, il controllo di quello che sta succedendo. Alcuni processi hanno bisogno di effettuare operazioni che non tutti potrebbero fare, per questo motivo nasce la problematica dell'implementare un meccanismo che gestisca i permessi di questi processi. La soluzione è stata creare due modelli di lavoro. Uno per i processi regolari che non hanno accesso a tutte le risorse hardware, detto User Mode e l'altro per dei processi particolari che hanno accesso a tutte le risorse detto Kernel Mode. Le chiamate di sistema fanno sì che i processi abbiano accesso alle funzionalità della kernel-mode. Per eseguire una chiamata di sistema un programma lancia un'istruzione trappola che salta nel kernel e aumenta il livello di priorità a kernel-mode. Dopo quest'istruzione trappola tutto ritorna come prima e il livello di privilegio ridiventa User Mode. Ovviamente dato che saltare da una modalità all'altra sarebbe un pericolo il Kernel ha una Trap table dove un numero è associato a ogni chiamata di sistema. Come fa il sistema operativo a riprendere il controllo della cpu, in modo da poter passare da un processo a un altro: ci sono due approcci, Uno cooperativo (aspettare per le chiamate di sistema) e uno non cooperativo (il sistema operativo prende il controllo). I processi cedono regolarmente la CPU effettuando chiamate di sistema come yield(). Le applicazioni trasferiscono il controllo al sistema operativo quando stanno facendo qualcosa di illegale (divisione per 0). L'utilizzo di questo approccio è spesso meno che ottimale: uno dei motivi potrebbe essere un processo cattivo che propositamente non fa nessuna chiamata di sistema. Per quanto riguarda l'altro approccio al sistema operativo serve che l'hardware sia collaborativo per realizzare un approccio non cooperativo. Timer interrupt: Durante la sequenza di avvio il sistema operativo inizializza un timer, il timer genera un'interruzione ogni tot tempo. Quando viene generata un'interruzione il processo in esecuzione viene arrestato, lo stato del programma viene salvato, un gestore di interruzioni preconfigurato nel sistema operativo viene eseguito. Questo Timer Interrupt consente al sistema operativo di riprendere l'esecuzione su una CPU. Quando si verifica un Timer Interrupt il sistema operativo si occupa di disabilitare le interruzioni date dal timer e si occupa di utilizzare degli schemi che impediscono l'accesso concorrente.

2 Scheduling

2.1 Introduzione allo scheduling

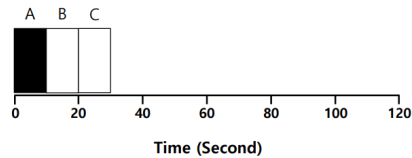
Abbiamo visto come viene implementato il context switch ma dobbiamo ancora discutere delle policy che si trovano al di sopra di questo meccanismo. Per farlo facciamo delle assunzioni che si discostano dalla realtà: (Queste assunzioni verranno modificate per capire meglio le politiche)

Ogni job (processo) ci mette lo stesso tempo a essere eseguito,
 Ogni job arriva nello stesso momento,
 Una volta iniziato il job dev'essere completato,
 Tutti i job utilizzano solo la cpu e non I/O,
 Il tempo di esecuzione di ogni job è noto.

Una metrica con cui giudicheremo le varie politiche di scheduling è il Turnaround Time che sarebbe quanto tempo il processo è rimasto nel sistema ovvero $T_{Turnaround} = T_{Completion} - T_{Arrival}$ con le assunzioni che abbiamo fatto adesso che ogni processo arriva a tempo 0 il Turnaround è uguale al tempo di completamento del processo. Un'altra metrica è “fairness” quanto equamente le risorse vengono divise tra i processi.

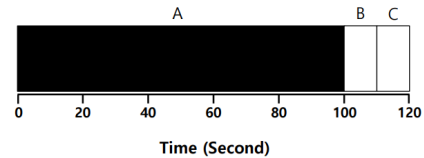
2.2 FIFO/First-In-First-Out o FCFS/First-Come-First-Served

Implementare una strategia fifo può essere conveniente solo con le nostre assunzioni, infatti se i processi non arrivassero tutti a tempo zero e avessero un tempo di esecuzione differente tra loro il turnaround medio si alzerebbe troppo e si crea quello che viene detto effetto convoglio, se arriva prima il processo più lungo, quelli dopo ne devono aspettare il completamento



$$\text{Average turnaround time} = \frac{10 + 20 + 30}{3} = 20 \text{ sec}$$

(a) Processi con tempo di esecuzione uguale



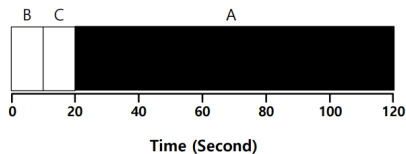
$$\text{Average turnaround time} = \frac{100 + 110 + 120}{3} = 110 \text{ sec}$$

(b) Processi con tempo di esecuzione differenti

Figure 1: FIFO/First-In-First-Out o FCFS/First-Come-First-Served

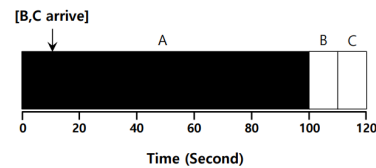
2.3 SJF/Shortest Job First

Come fa intuire il titolo viene eseguito prima il processo più corto poi il secondo più corto e così via. La criticità di questo è simile alla FIFO, in questo caso se è presente un solo processo che è anche il più lungo e ne viene iniziata l'esecuzione e poco dopo arrivano gli altri processi, nuovamente si ritrovano bloccati dal processo più lungo.



$$\text{Average turnaround time} = \frac{10 + 20 + 120}{3} = 50 \text{ sec}$$

(a) Arrivo in precedenza dei processi più piccoli



$$\text{Average turnaround time} = \frac{100 + (110 - 10) + (120 - 10)}{3} = 103.33 \text{ sec}$$

(b) Arrivo in precedenza del processo più grande

Figure 2: SJF/Shortest Job First

2.4 STCF/Shortest-Time-to-Completion-First

Questa politica è simile al SJF solo che non si basa sul tempo di esecuzione del processo ma sul tempo di esecuzione rimasto del processo.

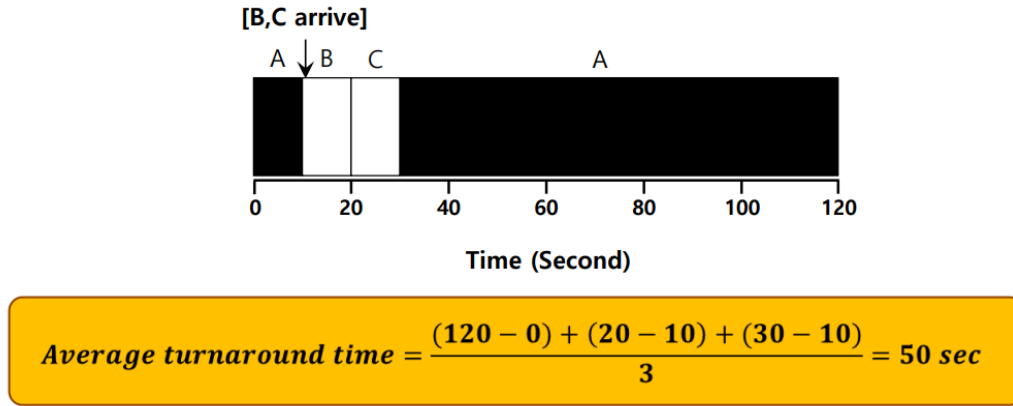


Figure 3: Miglioramento della politica precedente

Aggiungiamo un'altra metrica di giudizio che è il tempo di risposta, che consiste nel tempo che quel job è stato in coda prima di eseguire la prima istruzione ed è uguale a $T_{Response} = T_{Firstrun} - T_{Arrival}$

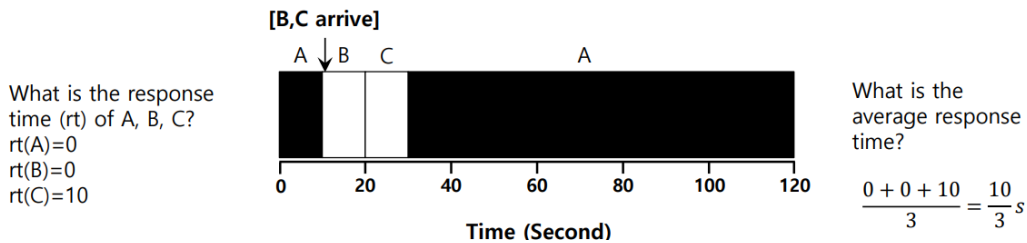


Figure 4: Calcolo del tempo di risposta

2.5 RR/Round Robin

Questa politica è basata su fette temporali, ovvero un job va in esecuzione per un certo quanto di tempo, successivamente viene mandato in esecuzione un altro job fino a che non c'è più nulla in coda. Questa politica è fair ma così facendo ottiene un turnaround pessimo.

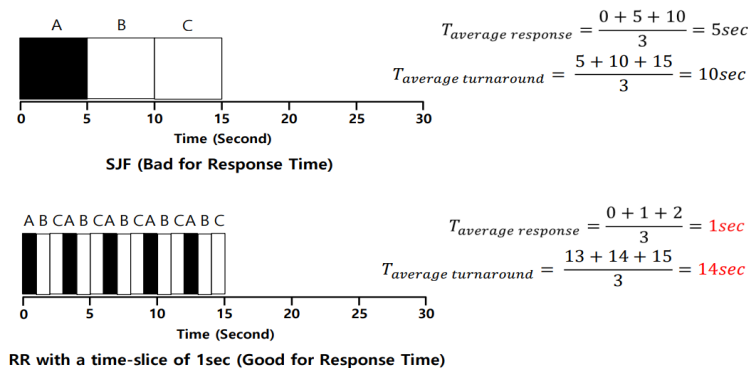


Figure 5: Ottimo tempo di risposta ma pessimo Turnaround Time

Questa politica sia con un quanto di tempo lungo che corto ha sempre delle criticità. Più il quanto di tempo è piccolo e più il costo del context switch dominerà le performance. Più il quanto di tempo è grande e peggiore sarà il tempo di risposta. Generalmente ogni politica fair avrà un cattivo turnaround time. Se volessimo incorporare anche l'I/O dovremmo modificare qualche assunzione, del tipo i processi possono usare I/O, Quando un job inizia una richiesta di I/O: Il job rimane bloccato e aspetta che l'I/O finisca, il gestore dovrebbe gestire un altro job sulla cpu. Quando l'I/O finisce viene sollevato un Interrupt e il sistema operativo modifica lo stato del processo da bloccato a pronto

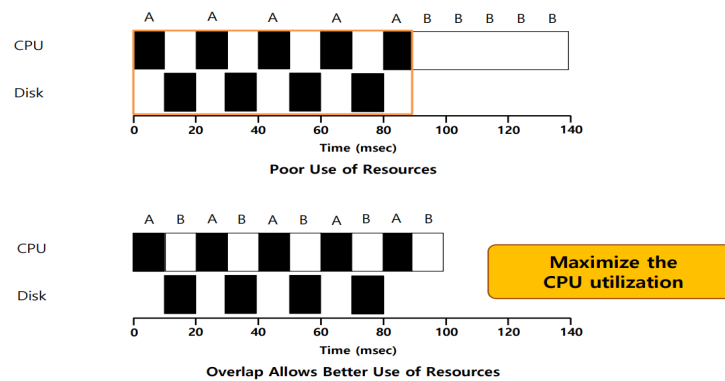


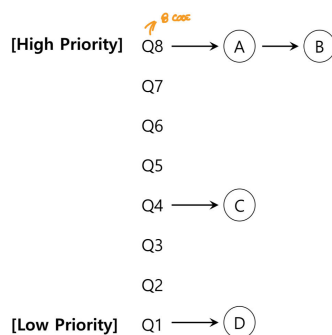
Figure 6: Gestione dell'I/O

2.6 Multi-Level Feedback Queue (MLFQ)

Con le altre politiche di scheduling abbiamo notato che per migliorare il turnaround time ottenevamo un tempo di risposta pessimo mentre per migliorare il tempo di risposta ottenevamo un turnaround time più elevato. Quello che tenta di fare il MLFQ è di ottimizzare il turnaround time e minimizzare il tempo di risposta, senza sapere a priori il tempo di esecuzione del processo. Questa politica di scheduling impara a run-time le caratteristiche dei job da eseguire, in modo tale da prendere delle decisioni migliori. MLFQ ha un numero di code distinte, ogni coda ha un livello di priorità.

RULE 1: se la $PRIORITA(a) > PRIORITA(b)$ viene eseguito a

RULE 2: se la $PRIORITA(a) = PRIORITA(b)$ a e b vengono eseguiti in RR



A e B vengono eseguiti in Round Robin, C e D non verranno mai eseguiti finchè A e B non terminano

With the rules we have just defined, A and B will be executed in a round robin fashion. Jobs C and D will never run, as long as A and B don't terminate

Figure 7: MLFQ e le regole appena definite

MLFQ cambia la priorità di un job osservandone il comportamento:

Se un job rilascia spesso la CPU per fare un'operazione di I/O gli lascia la priorità alta dato che probabilmente è un job interattivo e vogliamo che i job interattivi siano reattivi. Se un job utilizza la CPU per un lungo periodo di tempo gli riduce la priorità, questo perché si sta comportando come un job batch (es. avvio un calcolo la sera e la

mattina ne vedo il risultato).

Aggiungiamo delle regole al cambio di priorità:

RULE 3: Quando un job entra nel sistema gli viene attribuita la massima priorità

RULE 4a: Quando un job usa un intero quanto di tempo la sua priorità viene ridotta, questo perché potrebbe essere un job batch che richiede la cpu e non esegue operazioni di I/O

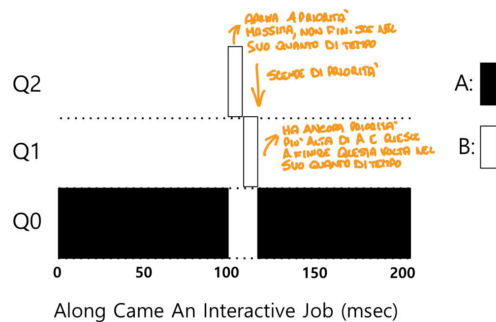
RULE 4b: Se un job rilascia la CPU prima che finisca il quanto di tempo la sua priorità rimane allo stesso livello, questo perché potrebbe essere un job interattivo

Esempio:

Il job A ha già avuto modo di dimostrare che non è un job interattivo

Il job B è un job interattivo che ha runtime 20ms

A è stato eseguito per una certa quantità di tempo, successivamente arriva B al tempo $T = 100ms$

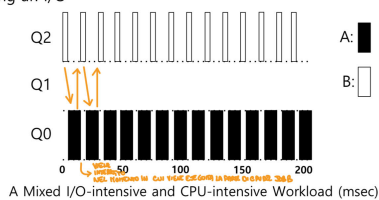


B arriva a massima priorità e non riesce a finire la sua esecuzione, gli viene scalato un livello di priorità, ha ancora priorità più alta di A e riesce a finire questa volta nel suo quanto di tempo

Figure 8: MLFQ con regole 4a e 4b

Comparando MLFQ con SJF (Short Job First) SJF doveva sapere a priori la lunghezza di ogni processo, con il set di regole che abbiamo messo MLFQ approssima SJF. La differenza è che MLFQ non gli serve sapere a priori la lunghezza del job, quando arriva un nuovo processo viene messo alla priorità massima, infatti se fosse un processo corto resterebbe alla massima priorità e lavorerebbe come SJF, se invece il processo fosse lungo, quest'ultimo, calerebbe di priorità.

- Job A: A long-running CPU-intensive job
- Job B: An interactive job that needs the CPU only for 1ms before performing an I/O



The MLFQ approach keeps an interactive job at the highest priority

Quest'esempio ci fa notare delle debolezze: -Starvation (morire di fame): Se ci fossero troppi job interattivi, i job a priorità più bassa potrebbero andare in starvation. -Ingannare lo scheduler: Un job malevolo, potrebbe, usare il 99% del quanto di tempo e successivamente fare una richiesta I/O in modo tale da non perdere di priorità

Figure 9: MLFQ con job batch e job interattivo

RULE 5 (Priority Boost): Dopo un certo periodo di tempo, sposto tutti i job a priorità massima ("gli do una seconda possibilità"), in poche parole dopo un po' un job batch verrebbe messo a RR con i job interattivi.

RULE 4: (Riscrivo 4a e 4b) Il job passa a un livello di priorità inferiore se viene eseguito per il tempo del quanto di tempo. Es: Ho un job malevolo e ho quanti di tempo da 10ms, utilizza 9ms e poi fa una richiesta di I/O quando verrà poi rieseguito, avrà a disposizione solo 1ms, dopodichè scenderebbe di priorità

MLFQ solitamente attribuisce ai livelli di priorità più alti quanti di tempo minori e viceversa. L'implementazione Solaris di MLFQ ha 60 code, e incrementa lentamente i quanti di tempo: Livello di priorità massimo: 20ms, Livello di priorità minimo qualche centinaia di ms. Il priority boost viene fatto ogni secondo.

2.7 The Linux Completely Fair Scheduling (CFS)

È la politica di scheduling che troviamo sulle macchine di adesso. Le sue principali caratteristiche sono: quanti di tempo variabili, supporta la priorità, controlla la priorità secondo un parametro di “gentilezza” (quanto il nostro processo debba essere gentile con gli altri processi) , utilizza delle strutture dati che sono più efficienti per ricerca, inserzione e cancellazione di un processo. I parametri base sono:

Virtual Runtime (Vruntime) : Denota per quanto tempo un processo è stato eseguito, ovvero la quantità in ms di cpu utilizzata dal processo. Incrementa proporzionalmente durante l’esecuzione di un job. Il CFS prenderà il processo avente runtime più basso possibile.

Sched_Latency : è il parametro che definisce la dimensione della finestra temporale da dividere in quanti di tempo (definendo questo parametro guadagniamo in termini di tempo di risposta), un valore tipico è di 48 ms. Il quanto di tempo di un processo = $\frac{\text{sched_latency}}{\text{numerodiprocessi}}$.

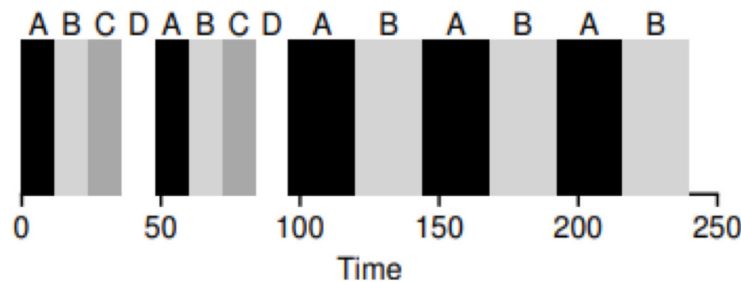


Figure 10: Esempio per capire il concetto di var

Ora c’è bisogno di fare delle precisazioni: il quanto di tempo minimo è di 6ms, infatti se riducessimo troppo il tempo dei quanti di tempo il SO impiegherebbe più tempo a fare il context switching che a eseguire i processi stessi. Il parametro di gentilezza va da -20 a +1 e viene tradotto in pesi numerici.

```
static const int prio_to_weight[40] = {
    /* -20 */ 88761, 71755, 56483, 46273, 36291,
    /* -15 */ 29154, 23254, 18705, 14949, 11916,
    /* -10 */ 9548, 7620, 6100, 4904, 3906,
    /* -5 */ 3121, 2501, 1991, 1586, 1277,
    /* 0 */ 1024, 820, 655, 526, 423,
    /* 5 */ 335, 272, 215, 172, 137,
    /* 10 */ 110, 87, 70, 56, 45,
    /* 15 */ 36, 29, 23, 18, 15,
};
```

Figure 11: Tabella dei pesi

- New timeslice formula

$$time_slice_k = \frac{weight_k}{\sum_{i=0}^{n-1} weight_i} \cdot sched_latency$$

- Simple Example

- Assign Process 'A' a nice value of -5 and process 'B' a nice value of 0.

Process	nice value	weight	Time slice
A	-5	3121	36 ms
B	0	1024	12 ms

- Weighting (Niceness)

- vruntime formula

- Calculate the actual run time. Scales it inversely by the weight of process.

$$vruntime_i = vruntime_i + \frac{weight_0}{weight_i} \cdot runtime_i$$

- Simple Example

Process	nice value	weight	Accumulated value
A	-5	3121	4/3 * runtime
B	0	1024	1 * runtime

Questa è la formula per il quanto di tempo, ovvero $\frac{PesoDelProcesso}{SommaDeiPesiDeiProcessi} * sched_latency$. In questo esempio il processo B verrà preferito dato che B avrà, dopo la prima esecuzione, Vruntime più basso e quindi B raggiungerà A. Per ovviare a questo viene aggiustato il calcolo del Vruntime che sale inversamente al peso del processo.

Una struttura basata su questo tipo di coda, sono gli alberi rosso-neri. L'ultima osservazione da fare è chiederci cosa succede in caso un processo voglia fare un'operazione di I/O. Innanzitutto bisogna evitare che gli altri processi aventi Vruntime basso monopolizzino la cpu, infatti quello che viene fatto è mettere al minimo il vruntime di tutti i processi.

3 Process API/Chiamate di sistema

3.1 Introduzione alle chiamate di sistema

Alcuni cenni prima dell'inizio

Per **creare un file.c** bisogna eseguire il comando **nano hello.c**. Per **salvare** bisogna premere **ctrl + o + invio**. Per **chiudere** il file che abbiamo appena aperto **ctrl + x**. Per **compilare** il file eseguiamo il comando **gcc -o nome_da_dare_all'eseguibile file.c**. Per **eseguire** il file aggiungiamo **./** davanti al nome dell'eseguibile. Di notevole importanza il comando **Man** che apre letteralmente il manuale di una chiamata di sistema.

3.2 La chiamata di sistema Fork()

La chiamata di sistema Fork(), fa un salto nel kernel, infatti sdoppia il processo che ha effettuato la chiamata di sistema. Il nuovo processo avrà la sua personale copia dello spazio degli indirizzi, dei registri e di PC. Avremo due processi identici, il quale, quello che ha chiamato la fork lo chiameremo processo padre e l'altro lo chiameremo processo figlio.

Listing 1: pl.c

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 int main(int argc, char *argv[]){
5     printf("hello - world - (pid:%d)\n", (int) getpid());
6     int rc = fork();
7     if (rc < 0) { // fork failed; exit
8         fprintf(stderr, "fork failed\n");
9         exit(1);
10    } else if (rc == 0) { // child (new process)
11        printf("hello, -I- am child - (pid:%d)\n", (int) getpid());
12    } else { // parent goes down this path (main)
13        printf("hello, -I- am parent - of -%d- (pid:%d)\n",
14            rc, (int) getpid());
15    }
16    return 0;}

```

Il processo che è stato creato dalla chiamata di sistema, non inizierà dalla prima istruzione del main ma dopo la fork. Sostanzialmente nell'esecuzione del programma quello che ci aspettiamo è che i processi genereranno dei valori diversi all'interno della variabile rc e in particolare che il processo figlio imbocchi il ramo con $rc = 0$. Se qualcosa fosse andato storto la fork avrebbe restituito un valore < 0 .

3.3 La chiamata di sistema Wait()

La chiamata di sistema Wait() sospende l'esecuzione del processo chiamante fino a quando uno dei suoi figli termina l'esecuzione.

Listing 2: p2.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 int main(int argc, char *argv[]){
6     printf("hello - world - (pid:%d)\n", (int) getpid());
7     int rc = fork();
8     if (rc < 0) { // fork failed; exit
9         fprintf(stderr, "fork - failed\n");
10        exit(1);
11    } else if (rc == 0) { // child (new process)
12        printf("hello , - I - am - child - (pid:%d)\n", (int) getpid());
13    } else { // parent goes down this path (main)
14        int wc = wait(NULL);
15        printf("hello , - I - am - parent - of - %d - (wc:%d) - (pid:%d)\n",
16            rc, wc, (int) getpid());
17    }
18    return 0;
19 }
```

Passiamo come parametro alla chiamata di sistema Wait() un puntatore nullable e ci aspettiamo che ci restituisca il PID del processo che ha cambiato il suo stato per primo.

3.4 Il programma analizzato in aula

Abbiamo creato un programma in aula che ci consenta di scrivere all'interno di un file

Listing 3: scrittura_file.c

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <string.h>
4 #include <unistd.h>
5 int main(){
6
7     char messaggio[]="ciao";
8     int fd=open("pippo.txt", O_CREAT | O_TRUNC | O_WRONLY, S_IRWXU);
9     write(fd, messaggio, strlen(messaggio));
10    close(fd);
11 }
```

In particolare: **O_CREAT** crea il file in caso non venga trovato, **O_TRUNC** se il file già esiste sarà troncato a 0, **O_WRONLY** indica che siamo interessati soltanto a scrivere il file e successivamente attribuisco al file i permessi **S_IRWXU** che indicano che il possessore del file può leggerlo, scriverlo e eseguirlo.

Se invece fossimo interessati a **leggere** il contenuto di `pippo.txt` dovremmo utilizzare la chiamata di sistema **`read()`** questa prende come primo parametro un file descriptor e come secondo un buffer che serve a immettere l'informazione che stiamo leggendo nella memoria.

Listing 4: `lettura_file.c`

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>
4 int main(){
5
6     int fd= open("pippo.txt", O_RDONLY, S_IRWXU);
7     int buffer=0;
8     read(fd, &buffer, sizeof(int));
9     printf("Buffer: %d\n", buffer);
10    close(fd);
11 }
```

3.5 La chiamata di sistema `Exec()`

La chiamata di sistema `Exec()` ci consente di eseguire altre applicazioni dal programma che stiamo utilizzando. Questa chiamata ha vari utilizzi i quali hanno diversa sintassi, in particolare abbiamo visto la `execv` che ha come primo parametro una stringa che fa riferimento al percorso dell'eseguibile che vogliamo andare a eseguire. La stringa che andiamo a passare come parametro sarà composta in questo modo: la posizione 0 fa riferimento al percorso dell'eseguibile e sarà seguita dagli argomenti che vogliamo passare all'eseguibile, l'ultimo elemento deve essere necessariamente `NULL`.

Listing 5: Chiamata di sistema `Exec()`

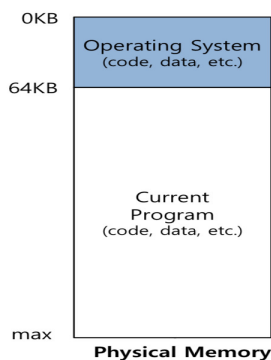
```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>
4 #include <string.h>
5 int main(){
6
7     int fd= open("pippo.txt", O_RDONLY, S_IRWXU);
8     int buffer=0;
9     read(fd, &buffer, sizeof(int));
10    printf("Buffer: %d\n", buffer);
11    close(fd);
12    char *myarg[2];
13    myarg[0]=strdup("ls"); //vado a mettere all interno di myarg[0]
14                          //un puntatore a qualcosa che sta dentro
15                          //l heap
16    myarg[1]=NULL;
17    execv(myarg[0], myarg);
18    printf("CIAO\n"); //questo non viene letto perche l'esecuzione
19                      //si ferma a execv
20 }
```

4 Spazio degli indirizzi

4.1 Introduzione allo spazio degli indirizzi

Abbiamo visto come la CPU possa essere virtualizzata in modo tale da ospitare più processi in esecuzione e vorremmo ottenere lo stesso effetto anche per la memoria. Parliamo di **virtualizzazione della memoria**, il sistema operativo virtualizza la memoria e fornisce uno spazio di memoria illusorio a ogni processo. Questo processo rende la memoria più efficiente in termini di spazio e tempo e garantisce l'isolamento per i processi e il sistema operativo. Infatti, non vogliamo che un processo anche se benevolo, cambi qualcosa, questo perché anche il minimo cambiamento potrebbe far crashare il sistema.

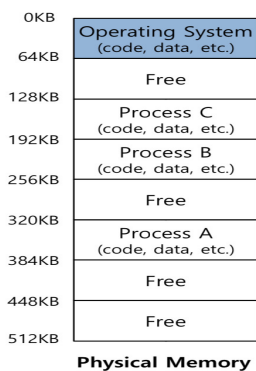
4.2 Gestione della memoria nei primi sistemi



Veniva caricato un solo processo in memoria, questo però aveva uno scarso utilizzo e efficienza. Come pro aveva delle prestazioni ineguagliabili per l'accesso in memoria

Figure 12: Gestione della memoria nei primi sistemi

4.3 Gestione della memoria con multiprogrammazione e Time Sharing



Vengono caricati più processi in memoria, si passa da un processo all'altro che viene eseguito per un piccolo quanto di tempo, questo incrementa l'utilizzo e l'efficienza. Il suo contro è che vi si possono causare degli errati accessi in memoria da parte dei processi

Figure 13: Gestione della memoria con multiprogrammazione e Time Sharing

4.4 Gestione della memoria con lo spazio degli indirizzi

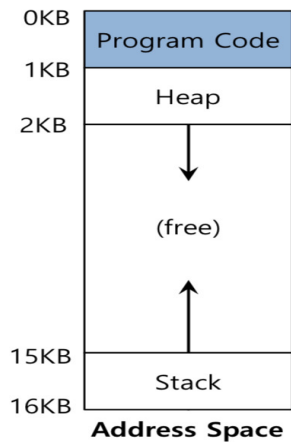


Figure 14: Gestione della memoria con lo spazio degli indirizzi

Come abbiamo detto il sistema operativo astrae la memoria fisica. Lo spazio degli indirizzi contiene tutto riguardo il processo in esecuzione, che consiste in codice del programma, heap, stack...

Ogni processo avendo il proprio spazio degli indirizzi avrà l'illusione di avere uno spazio privato sulla memoria, invece sarà compito del sistema operativo abbinare gli indirizzi virtuali agli indirizzi fisici

Program Code: è la parte dove vivono le istruzioni

Heap: è dove viene allocata dinamicamente la memoria (malloc (C), new (programmazione orientata agli oggetti))

Stack: dove vengono immagazzinati i valori e gli indirizzi di ritorno, contiene variabili locali e argomenti.

Un'altro esempio che ci permette di comprendere lo spazio degli indirizzi è questo codice, infatti lanciandolo diverse volte sembrerà che il programma utilizzi sempre gli stessi indirizzi, questo perchè sono virtuali.

Listing 6: Spazio degli indirizzi

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main(int argc, char *argv[]) {
4     printf("location of code: %p\n", (void *) main);
5     printf("location of heap: %p\n", (void *) malloc(1));
6     int x = 3;
7     printf("location of stack: %p\n", (void *) &x);
8     return x;
9 }
```

5 Traduzione degli indirizzi

5.1 Introduzione alla traduzione degli indirizzi

Come per la virtualizzazione della CPU, vogliamo ottenere un meccanismo efficiente e controllabile per la virtualizzazione della memoria, anche qui sarà necessario il supporto hardware. Questo supporto consiste nella trasformazione da indirizzo virtuale a indirizzo fisico.

5.2 Assunzioni per iniziare l'analisi della traduzione degli indirizzi

Lo spazio degli indirizzi di un processo deve essere collocato contiguamente nella memoria fisica.

La dimensione dello spazio degli indirizzi è più piccola della dimensione della memoria fisica

Ogni spazio degli indirizzi ha la stessa dimensione

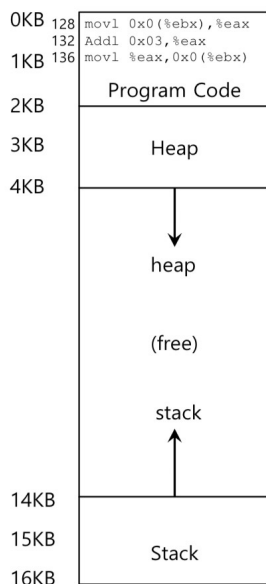
5.3 Applicazione che accede alla memoria

In codice C: viene caricato il valore dalla memoria, viene incrementato di tre e memorizzato nella memoria

```
1 void func() {  
2     int x;  
3     ...  
4     x= x+3;  
5 }
```

In Assembly: viene caricato il valore dell'indirizzo dentro il registro eax, viene aggiunto 3 al registro eax, viene caricato il valore dentro eax nella memoria

```
1 128 : movl 0x0(%ebx), %eax ; load 0+ebx into eax  
2 132 : addl $0x03, %eax ; add 3 to eax register  
3 136 : movl %eax, 0x0(%ebx) ; store eax back to mem
```



Recupera (Fetch) l'istruzione all'indirizzo 128

Esegue l'istruzione (carica dall'indirizzo 15KB)

Recupera l'istruzione all'indirizzo 132

Esegue l'istruzione

Recupera l'istruzione all'indirizzo 136

Esegue l'istruzione (Carica all'indirizzo 15KB)

Figure 15: Applicazione che accede alla memoria

5.4 Registri Base e Bound

Nel registro **Base** c'è l'indirizzo iniziale della memoria fisica, nel registro **Bound** c'è la dimensione dello spazio degli indirizzi che sto tenendo in considerazione. Questi due sono due "pezzi" hardware fisici.

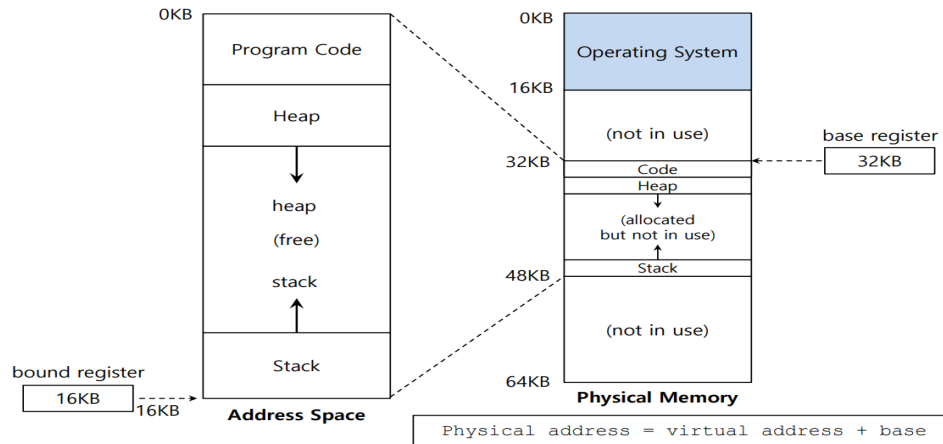


Figure 16: Registri Base e Bound

Se consideriamo solamente il registro Base ci sarebbe un problema, ovvero non vogliamo che qualcuno acceda ad un determinato indirizzo solamente perchè conosce questa formuletta. A tal proposito viene aggiunto il registro Bound infatti grazie ad esso controllo se la richiesta di traduzione dell'indirizzo non sia più grande della dimensione dello spazio degli indirizzi allocato per il processo.

128 : movl 0x0(%ebx), %eax

128 → 128 + 32768 = 32896

- Fetch instruction at address 128

128 → 128 + 32768 = 32896

- Execute this instruction

- Load from address 15KB

15K → 15K + 32K = 47K

Ora capiamo meglio l'esempio di prima:

Recupera (Fetch) l'istruzione all'indirizzo 128 (traduzione)

Esegue l'istruzione (carica dall'indirizzo 15KB)

Completamento da parte del sistema operativo Il supporto che ci fornisce l'hardware non è male, ma va completato con l'implementazione con il sistema operativo. Il sistema operativo deve prendere dei provvedimenti per implementare l'approccio base-and-bound. Ci sono tra punti di giunzione critici: quando un processo inizia l'esecuzione, bisogna trovare uno spazio degli indirizzi sufficientemente grande nella memoria fisica, quando un processo termina, va reclamata la memoria e infine quando si verifica il context switch, in cui bisogna salvare e caricare i valori dei registri base e bound.

5.5 Primo problema: quando un processo inizia l'esecuzione

Il sistema operativo deve trovare uno spazio per lo spazio degli indirizzi: viene quindi istanziata una free list, ovvero, una lista di range della memoria fisica che non sono in uso. Solitamente è da slot di 16KB

5.6 Secondo problema: quando un processo viene terminato

Il sistema operativo rimette nella free list la memoria.

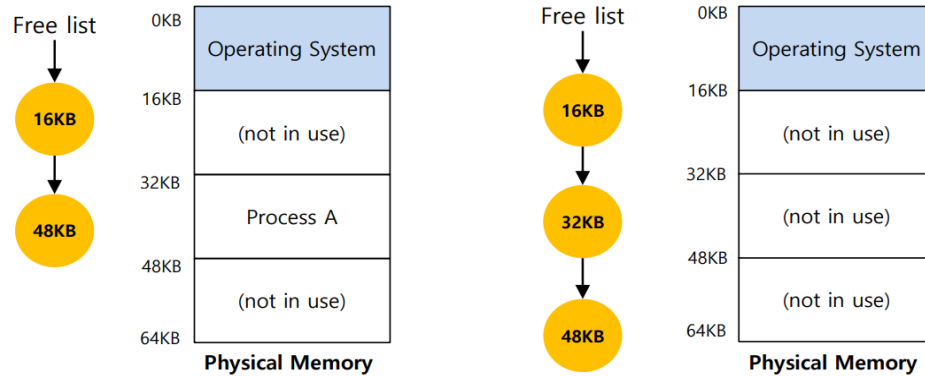


Figure 17

5.7 Terzo problema: quando si verifica il context switch

Il sistema operativo salva i valori dei registri Base e Bound

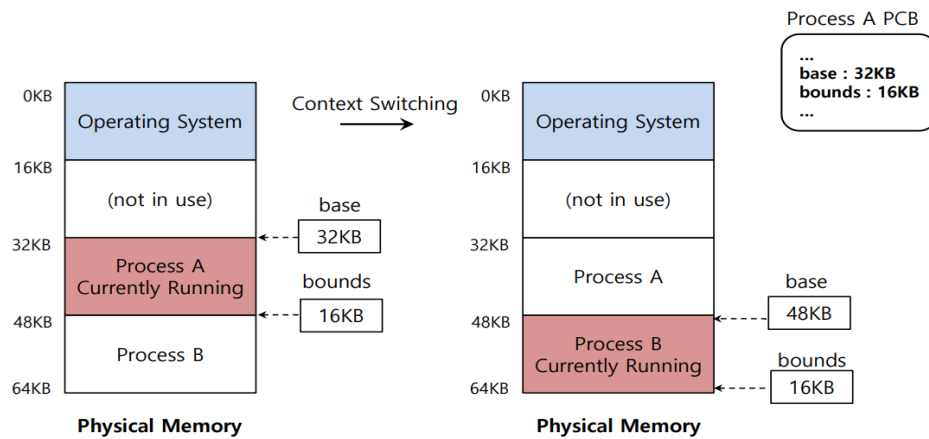


Figure 18

